

Technical Study Guide & Q&A Preparation: Assignment 1 Part 2

This guide is an exhaustive technical reference designed to prepare you for a technical Q&A session. It covers the *What*, *Why*, and *How* of every major system in the renderer.

1. Vulkan Fundamentals & Initialization

The Vulkan Hierarchy

1. **VkInstance**: The connection between your application and the Vulkan library. It enables **Instance Layers** (like Validation Layers) and **Instance Extensions** (like **VK_KHR_surface** for windowing).
2. **VkPhysicalDevice**: Represents the actual hardware (GPU). We query properties (Name, VRAM size) and features (Geometry Shader support, etc.) to select the best one.
3. **VkDevice (Logical Device)**: The logical interface to the GPU. This is where we create resources (buffers, images) and queues.

volk: The Meta-Loader

Q: What is volk and why do we use it? A: Vulkan is a dynamically loaded library. By default, you have to manually load function pointers (like **vkCreateDevice**) using **vkGetInstanceProcAddr**. **volk** automates this. It loads the entry points for us, avoiding the overhead of the standard loader's dispatch tables and simplifying cross-platform setup.

Device Selection Logic (**score_device**)

We score devices to pick the best one:

- **Discrete GPU** (e.g., GTX 1660 Ti) gets +500 points.
- **Integrated GPU** (e.g., Intel HD) gets +100 points.
- **Critical Checks**:
 - Must support **Vulkan 1.3** (for Dynamic Rendering).
 - Must support **VK_KHR_swapchain** extension.
 - Must have a **Graphics Queue** and a **Present Queue**.

Queues & Families

Vulkan exposes "Queue Families". A family has specific capabilities (Graphics, Compute, Transfer).

- We prefer a single family that supports **both** Graphics and Presentation (less synchronization overhead).
 - If separate, we must manage ownership transfers or use **VK_SHARING_MODE_CONCURRENT**.
-

2. Swapchain & Presentation

The Swapchain (**VkSwapchainKHR**)

A chain of images used for display. It bridges the creation of the image (GPU) and the display of the image (Monitor).

Q: Why **VK_FORMAT_B8G8R8A8_SRGB?** A:

1. **B8G8R8A8**: A standard 32-bit color format (Blue, Green, Red, Alpha). Widely supported.
2. **SRGB**: Crucial. It tells the GPU that the data we write to this image is in the sRGB color space. The GPU's hardware output merger will automatically apply **Gamma Correction** (converting linear colors to non-linear sRGB) when writing to this image. This saves us from doing **pow(color, 1/2.2)** manually.

Presentation Modes

- **VK_PRESENT_MODE_FIFO_KHR**: The "V-Sync" mode. Hard requirement by the spec. The queue waits for the vertical blank. No tearing, but caps FPS at refresh rate.
 - **VK_PRESENT_MODE_MAILBOX_KHR**: "Triple Buffering". Render as fast as possible; if the display isn't ready, overwrite the queued image with a newer one. Lowest latency, no tearing, high power usage.
 - **VK_PRESENT_MODE_IMMEDIATE_KHR**: No V-Sync. Tearing possible.
-

3. Memory Management

Memory Types

Vulkan exposes memory explicitly. We use **VMA (Vulkan Memory Allocator)** to manage this, but you must understand the underlying types:

1. **Device Local (`VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT`):**
 - High-speed VRAM on the GPU card.
 - **Usage:** Vertex Buffers, Index Buffers, Textures (Images), Offscreen Framebuffers.
 - **Access:** CPU usually *cannot* write here directly.
2. **Host Visible (`VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT`):**
 - System RAM (or a special mapped bar) accessible by both CPU and GPU.
 - **Usage:** Staging Buffers, Uniform Buffers.
 - **Coherent (`HOST_COHERENT_BIT`):** Writes assume automatic visibility (no `vkFlushMappedMemoryRanges` needed).

Staging Buffers

Q: How do we get a mesh into VRAM if the CPU can't write to it? A: The Staging Buffer Dance:

1. CPU allocates a **Staging Buffer** in Host Visible memory.
2. CPU `memcpy` data (Vertices) into the Staging Buffer.
3. Command Buffer records `vkCmdCopyBuffer`: Copy from Staging Buffer -> GPU Device Local Buffer.
4. Submits command.
5. Wait for completion, then delete Staging Buffer.

4. The Rendering Pipeline

We use **Dynamic Rendering** (Vulkan 1.3), which removes the need for `VkRenderPass` and `VkFramebuffer` objects.

Pipeline Stages (`create_triangle_pipeline`)

1. **Vertex Input:** Defines the layout of data in the buffer (Stride=12 bytes for vec3 pos).
2. **Input Assembly:** Group vertices into Triangles (`VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST`).
3. **Vertex Shader:** Transforms Local Pos -> Clip Space (`gl_Position`).
4. **Rasterization:**
 - Culls back-facing polygons (`VK_CULL_MODE_BACK_BIT`).
 - Converts geometry into fragments (pixels).
5. **Fragment Shader:** Computes color per pixel.
6. **Depth Test:** Checks `depthBuffer`. If `currentDepth <= storedDepth`, keep the pixel.
7. **Color Blending:**
 - **Opaque:** Output color replaces existing color.
 - **Transparent:** `NewColor * Alpha + OldColor * (1-Alpha)`.

Descriptor Sets

"Descriptors" are handles to resources (Buffers, Textures) that shaders can access.

- **Set 0: Scene Uniforms:** `binding = 0`. Contains Matrices (View, Proj) and Camera Pos.
 - *Type:* `VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER`. Fast for small, read-only data.
- **Set 1: Material Textures:** `binding = 0, 1, 2`. Albedo, Roughness, Metalness.
 - *Type:* `VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER`. It combines the Image View (data) with a Sampler (filtering).

5. Synchronization (The "Hard Part")

The GPU is an asynchronous beast. It might execute commands out of order unless we explicitly constrain it.

1. Fences (CPU-GPU Sync)

- **Usage:** `vkWaitForFences`.
- **Scenario:** The CPU wants to record Frame N. It must verify that the GPU is finished reading the Command Buffers from Frame N-1 (or N-2). `frameDone` fences ensure we don't overwrite a command buffer currently in flight.

2. Semaphores (GPU-GPU Sync)

- **Usage:** `VkSubmitInfo` wait/signal semaphores.
- **Scenario:**

- **imageAvailable**: The Swapchain signals this when it has an image ready. The **rendering commands** wait on this semaphore at the **VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT** stage.
- **renderFinished**: The rendering commands signal this when done. The **Presentation Engine** waits on this before showing the image.

3. Pipeline Barriers (Command-Command Sync)

Used within a single command buffer to handle data dependencies and layout transitions.

Example: The Scene UBO Barrier

```
lut::buffer_barrier( ...,
    VK_PIPELINE_STAGE_TRANSFER_BIT, VK_ACCESS_TRANSFER_WRITE_BIT, // Source: We just computed/uploaded
    data
    VK_PIPELINE_STAGE_VERTEX_SHADER_BIT, VK_ACCESS_UNIFORM_READ_BIT // Dest: The shader is about to
    read it
);
```

Translation: "Don't let the Vertex Shader read this buffer until the Transfer stage is completely finished writing to it."

Example: Image Layout Transition Images must be in the correct "Layout" for efficiency/correctness.

- **Writing**: **VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL**.
- **Reading (Sampling)**: **VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL**.
- **Presenting**: **VK_IMAGE_LAYOUT_PRESENT_SRC_KHR**. A barrier performs this transition and ensures memory caches are flushed.

6. Physically Based Rendering (PBR)

We use the **Cook-Torrance Specular BRDF**.

The Equation

$$L_o = \int (k_d \frac{c}{\pi} + k_s D(h) F(v, h) G(l, v, h) / (4(n \cdot l)(n \cdot v))) L_i (n \cdot l) d\omega$$

- **Diffuse Term (\$k_d\$)**: Lambertian reflection. Light scatters equally.
- **Specular Term (\$k_s\$)**: Microfacet reflection.
 - **D (Distribution): Beckmann**. How lots of micro-mirrors are aligned. High roughness = spread out D = blurry highlight.
 - **F (Fresnel): Schlick**. Light reflects more at grazing angles. Everything is 100% reflective at 90 degrees.
 - **G (Geometry): Smith/Cook-Torrance**. Microfacets block each other (self-shadowing).

Textures

1. **Albedo/BaseColor**: The diffuse color (dielectrics) or specular color (metals).
2. **Roughness**: 0.0 (Smooth mirror) to 1.0 (Matte chalk). Controls the 'D' term.
3. **Metalness**: 0.0 (Dielectric) or 1.0 (Metal). Metals have no diffuse component (\$k_d = 0\$) and colored specular (\$F_0\$ = Albedo).

7. Post-Processing Architecture

Why render to an offscreen buffer first?

The Problem: HDR

Real-world light has infinite range. The sun is thousands of times brighter than a lamp. If we write directly to the Swapchain (Standard 0.0 - 1.0 range), any value > 1.0 is clamped to 1.0 (White). This loses detail and looks "flat".

The Solution: 16-bit Float + Tone Mapping

1. **Pass 1**: Render to **VK_FORMAT_R16G16B16A16_SFLOAT**. This buffer can store values like 50.0 or 1000.0 without clamping.
2. **Pass 2 (Resolve)**: Sample that texture. Apply **Tone Mapping**.
 - **Reinhard Operator**: $Color = \frac{Color}{1.0 + Color}$.
 - Example: Input 1000.0 -> Output 0.999 (White, but detailed gradient approaching it).
3. **Gamma Correction**: Finally, convert Linear Space -> sRGB Space for the monitor. (Done by hardware sRGB swapchain).

8. Common Q&A Scenarios

Q: Why is my screen black?

- Camera looking wrong way?
- Forgot `BeginRendering`?
- Forgot to transition image to `COLOR_ATTACHMENT_OPTIMAL`?
- Shader outputting 0 alpha?

Q: Why do I see tearing?

- Present mode is likely `IMMEDIATE` or `MAILBOX`. Switch to `FIFO`.

Q: Why are my textures upside down?

- Vulkan's Y coordinate is down. OpenGL's is up. GLM projection matrices usually need a flip: `projection[1][1] *= -1.f;`

Q: The validation layer says "Descriptor Set not bound"?

- You called `vkCmdDraw` but the pipeline layout expects a descriptor set that wasn't bound *with a compatible layout* for that specific pipeline. (This was the bug we fixed!).

Q: Why is the scene so bright/washed out?

- **Double Gamma:** You applied `pow(color, 1/2.2)` in the shader AND used an `_SRGB` swapchain format. Remove the shader code.

9. Deep Dive: Synchronization Barriers

In `rendering.cpp`, we use explicit barriers for correctness. Here is the exact logic:

Purpose	SrcStage	SrcAccess	DstStage	DstAccess	Layout Change
Pass 1: Offscreen Ready	TOP_OF_PIPE	NONE	COLOR_ATTACHMENT_OUTPUT	WRITE	UNDEFINED -> COLOR_ATTACHMENT_OPTIMAL
Pass 1: Depth Ready	TOP_OF_PIPE	NONE	EARLY/LATE_FRAGMENT	WRITE	UNDEFINED -> DEPTH_STENCIL_OPTIMAL
Pass 2: Input Read	COLOR_ATTACHMENT_OUTPUT	WRITE	FRAGMENT_SHADER	READ	COLOR_ATTACHMENT_OPTIMAL -> SHADER_READ_ONLY
Pass 2: Output Ready	TOP_OF_PIPE	NONE	COLOR_ATTACHMENT_OUTPUT	WRITE	UNDEFINED -> COLOR_ATTACHMENT_OPTIMAL
Presentation	COLOR_ATTACHMENT_OUTPUT	WRITE	BOTTOM_OF_PIPE	NONE	COLOR_ATTACHMENT_OPTIMAL -> PRESENT_SRC

Q: Why TOP_OF_PIPE? A: It implies the "start of the command processing". We use it when we don't care about what happened before (because `UNDEFINED` layout discards data anyway).

Q: Why BOTTOM_OF_PIPE? A: It implies "end of all commands". Used for Presentation because the Semaphore `renderFinished` handles the actual synchronization with the Display Engine; the barrier just handles the layout transition.

10. Deep Dive: Shader Logic

PBR Implementation (`default.frag`)

- **Beckmann Distribution:**

```
float num = exp( (NdotH2 - 1.0) / (alpha2 * NdotH2) );
float den = PI * alpha2 * NdotH2 * NdotH2;
return num / den;
```

Why Beckmann? It's a standard choice for microfacet models, similar to GGX but with different tail characteristics.

- **Masking (G1):**

```
(2.0 * NdotH * NdotV) / VdotH;
```

Basic Cook-Torrance geometric attenuation factor.

Debug Shaders

- **debug_mip.frag:** Uses `textureQueryLod(uTex, coords)` to get the current mipmap level, then assigns a specific color (Red=0, Green=1, Blue=2...) to visualize texture streaming/minification.
- **debug_depth.frag:** Outputs `gl_FragCoord.z`.
 - *Note:* This is non-linear. To see detail, we often need to linearize it: $\frac{2 \cdot n \cdot f}{(f + n - z \cdot (f - n))}$.
- **debug_deriv.frag:** Uses `dFdx(v2fPos)` and `dFdy` to show the rate of change of depth/position, useful for detecting edge discontinuities.